
Harness를 설계하는 법

챗봇과 싸우지 않고, 에이전트가 일할 환경을 설계하는 방법

게임플랫폼 1팀 라승수

시작하며

01

챗봇과 말씨름

맥락 충돌과 반복 설명

02

도구들의 수렵

자동완성·채팅 UI·터미널 에이전트

03

Harness

규칙·컨텍스트·권한·검증·상태 기록

개발자의 핵심 역량: AI에게 할 말 → AI가 안전하게 일할 환경

전체 목차

- | | | |
|----|-----------------|------------------------|
| 01 | 코딩은 어디로 가고 있는가 | 개발자의 숙련 이동 |
| 02 | 왜 Claude Code인가 | AI 코딩 도구의 구조 수렴 |
| 03 | AI 시대의 개발 방법론 | TDD·SDD·Spec-first 재부상 |
| 04 | Harness | Prompt 이후의 에이전트 운영 구조 |
| 05 | 한계와 실패 패턴 | 긴 작업·보안·신뢰 문제 |
| 06 | 멀티 에이전트 | 역할과 컨텍스트 분리 |
| 07 | 워크플로우와 도구 | 명령어·게이트·워크트리·이슈 |
| 08 | 제작 과정 | 발표 제작 파이프라인 해부 |
| 09 | 다음에 해야 할 일 | 개인과 팀의 첫 변화 |

CHAPTER 01

코딩은 사라지는가

기계어 → 어셈블리

기계어

```
10111000 00000001 00000000 00000000 00000000
```

어셈블리

```
MOV EAX, 1
```

기계어를 모르면 진짜 개발자 아님

어셈블리 → C/Pascal

어셈블리

```
section .data
    msg db "Hello World", 10

section .text
    global _start

_start:
    mov rax, 1
    mov rdi, 1
    mov rsi, msg
    mov rdx, 12
    syscall
    mov rax, 60
    mov rdi, 0
    syscall
```

C/Pascal

```
#include <stdio.h>

int main() {
    printf("Hello\n");
    return 0;
}
```

어셈블리를 모르면 진짜 개발자 아님

C → Java

C

```
#include <stdio.h>
#include <stdlib.h>

typedef struct {
    int x;
    int y;
} Point;

int main() {
    Point* p = malloc(sizeof(Point));
    p->x = 10;
    p->y = 20;
    printf(
        "Point: %d, %d\n",
        p->x,
        p->y
    );
    free(p);
    return 0;
}
```

Java

```
class Point {
    int x;
    int y;

    Point(int x, int y) {
        this.x = x;
        this.y = y;
    }
}

public class Main {
    public static void main(String[] args) {
        Point p = new Point(10, 20);
        System.out.println(
            "Point: " + p.x + ", " + p.y
        );
        p = null;
    }
}
```

메모리 직접 관리 없는 개발은 장난

Java → Python

Java

```
public class HelloWorld {  
    public static void main(String[] args) {  
        System.out.println("Hello, World!");  
    }  
}
```

Python

```
print("Hello, World!")
```

보일러플레이트 없는 언어는 장난감

AI 개발

직접 작성

대상 코드 탐색

수정 지점 선택

테스트 작성

결과 검토

AI 개발

대상 함수 리팩터링
테스트 코드 작성
검증 결과 정리

자연어 지시는 개발이 아님

직접 하던 일의 감소

시대	전환	위임한 것
1950s	기계어 → 어셈블리	이진 코드
1960s	어셈블리 → C	명령어 최적화
1990s	C/C++ → Java	메모리 해제
2000s	언어 → 프레임워크	보일러플레이트
2010s	온프레미스 → 클라우드	서버 운영·확장
2020s	직접 코딩 → AI 에이전트	코드 작성 일부

개발자가 끝까지 직접 붙들 일의 이동 위치

숫자로 보는 변화

25%

YC W25

95% 이상 AI 코드베이스

100만 줄

OpenAI Codex

수동 코드 없이 프로젝트 완성

3개월+

Meta

직접 타이핑 중단 사례

신호	규모	의미
YC W25	25%	AI 작성 비중
Codex 팀	100만 줄	수동 코딩 감소
Meta 사례	3개월+	역할 변화

문서의 코드화

파일	역할	에이전트 기준
README	사람의 입구	프로젝트 방향
CLAUDE.md	세션 운영	작업 방식
AGENTS.md	역할과 금지선	우선순위
Cursor Rules	도구별 규칙	편집 기준
copilot-instructions	반복 기준	코드 생성 습관

컨텍스트를 설계하는 능력

이전 역할

코드 직접 작성

버그 수정

문서 사후 정리

새 역할

문제 분해

컨텍스트 선택

수용 기준

검증 gate

권한 경계

BASICS

타이핑 감소, 시스템 판단 책임

인증·동시성·DB·캐시·보안·비용·배포

Architecture

Security

Operations